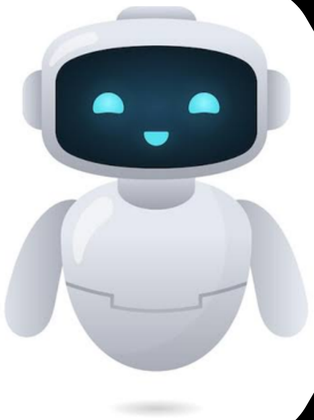
IIT GOA




# ROS 2

## LECTURE -2



Presented by **AAYUSH VATS**

# RECAP

- DOCKER.
- ROS2 SETUP.
- ABOUT NODES, TOPICS, MESSAGES, SERVICES, ACTIONS, LAUNCH, PARAMETERS
- TURTLESIM
- RQT
- RQT_CONSOLE

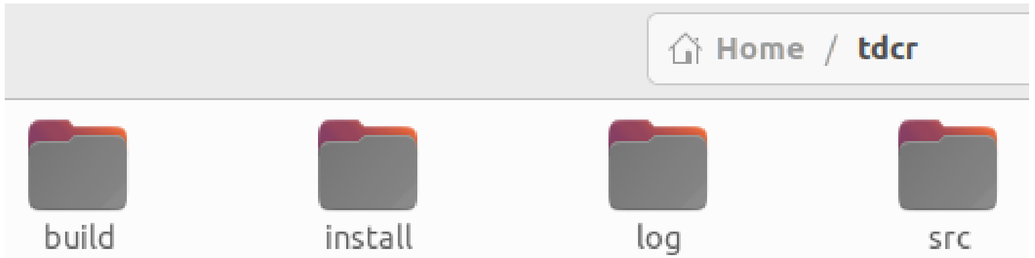
# USE OF COLCON & BUILDING WORKSPACE

- “COLCON” is the **automated** approach to build a set of packages in their topological order one by one. In addition to it, the tool generates the environment setup scripts for build workspace.

- ***sudo apt install python3-colcon-common-extensions*** is the command to install colcon, it is a necessary tool to install if you want to work on ROS ecosystem.

- A ROS “WORKSPACE” is a directory witha particular structure. It contains a subdirectory named as *src*. Inside it we can locate the ROS packages.

- *Build, Install* and *Log* are the other three subdirectories.




- To create a workspace on your terminal use the command:
  $ **mkdir -p ~/ros2_ws/src**
  $ **cd ~/ros2_ws**

- Building the workspace is the next step. Use the Following command in the terminal:
  $ **colcon build --symlink-install.**
  adding “--symlink-install” is completely optional. This argument ensures that you need not rebuild everytime you make change in your python code.

# BUILDING WORKSPACE

- Other useful colcon build command arguments:
  **--packages-select:** selects the specific package you want to build.
  **--event-handler console_direct+:** shows console output.

- Once you see the 4 subfolders into your workspace, the next step is to source your workspace inside the root. For this you have two options:
  1.) In the terminal use **source install/local_setup.bash** after every time you colcon build.
  2.) Open your .bashrc file and below the line "***source /opt/ros/humble/setup.bash"*** add " ***source install/local_setup.bash".*** Next close the terminal and open it again.

- Inside your **src** you can also run projects that other people have made public on github. It is something similar to downloading apps on your laptop/phones the process is called **cloning**. Use the command:
  ***git clone <link_to_github_repository>.***

# CREATING A PACKAGE

- A ROS package is an organizational unit for your code. If you want your code to be installed or run into other systems it has to be organized in form of a package.
- A package uses ament as its build system and colcon as its build tool.
- You have 2 options to make a package in ROS 2.

1.) **CMake:** Usually this package build has *CMakeLists.txt, package.xml, include/<package_name>*. This package is build when a user want to make **srv files, msg, cpp codes** into his package.

2.) **Python:** Usually this package build has *package.xml, resource/<package_name>, setup.cfg, setup.py,* and a <package name> inside which *__init__.py* is present. This package is build when the user wants to make python codes.

- To build a cmake package, first navigate yourself inside the src of your ROS workspace.
- Next write the command for creating a cmake package:
  **ros2 pkg create --build-type ament_cmake --license Apache-2.0 <package_name>**
- Next write the command for creating a python package:
  **ros2 pkg create --build-type ament_python --license Apache-2.0 <package_name>**
- Go back to the workspace build location and build your package using colcon build. if you are building a python package try to add --symlink-install as the argument.

# CREATING A PACKAGE

**CMake package message**

```
going to create a new package
package name: my_package
destination directory: /home/user/ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['<name> <email>']
licenses: ['TODO: License declaration']
build type: ament_cmake
dependencies: []
node_name: my_node
creating folder ./my_package
creating ./my_package/package.xml
creating source and include folder
creating folder ./my_package/src
creating folder ./my_package/include/my_package
creating ./my_package/CMakeLists.txt
creating ./my_package/src/my_node.cpp
```

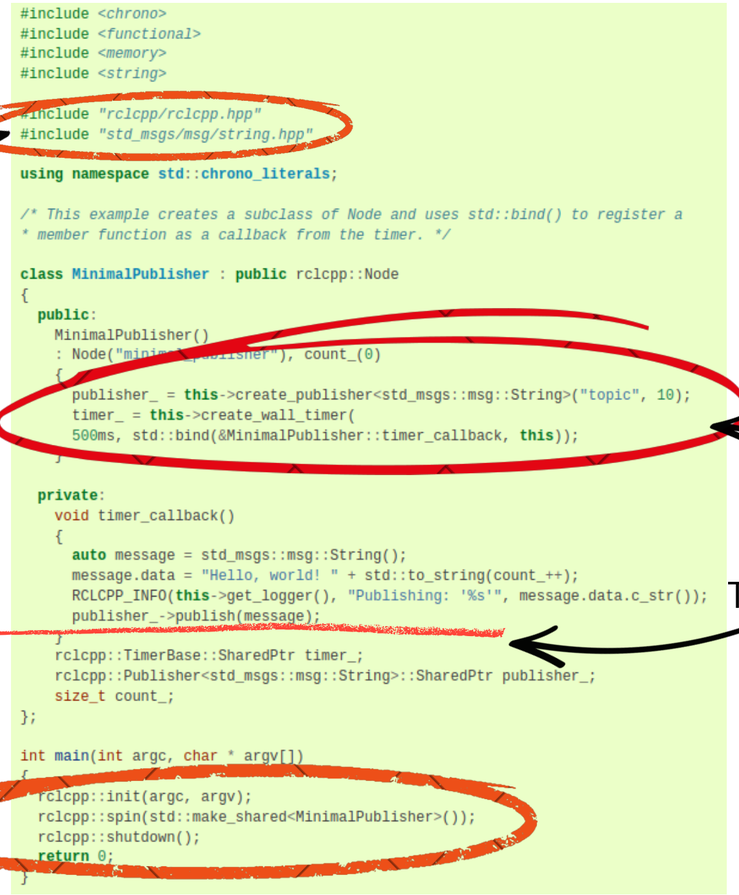
**Python package message**

```
going to create a new package
package name: my_package
destination directory: /home/user/ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['<name> <email>']
licenses: ['TODO: License declaration']
build type: ament_python
dependencies: []
node_name: my_node
creating folder ./my_package
creating ./my_package/package.xml
creating source folder
creating folder ./my_package/my_package
creating ./my_package/setup.py
creating ./my_package/setup.cfg
creating folder ./my_package/resource
creating ./my_package/resource/my_package
creating ./my_package/my_package/__init__.py
creating folder ./my_package/test
creating ./my_package/test/test_copyright.py
creating ./my_package/test/test_flake8.py
creating ./my_package/test/test_pep257.py
creating ./my_package/my_package/my_node.py
```

# CREATING A CPP PUBLISHER

- As Discussed in Lecture 1, **Nodes** are the executable processes that communicate over ROS graph. They pass information over a **TOPIC**.

Headers: **rclcpp** uses most common pieces for ROS2 CPP, **std_msgs/msgs/string** includes the built-in message type you will use to publish data.



```cpp
#include <chrono>
#include <functional>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;

/* This example creates a subclass of Node and uses std::bind() to register a
* member function as a callback from the timer. */

class MinimalPublisher : public rclcpp::Node
{
  public:
    MinimalPublisher()
    : Node("minimal_publisher"), count_(0)
    {
      publisher_ = this->create_publisher<std_msgs::msg::String>("topic", 10);
      timer_ = this->create_wall_timer(
      500ms, std::bind(&MinimalPublisher::timer_callback, this));
    }

  private:
    void timer_callback()
    {
      auto message = std_msgs::msg::String();
      message.data = "Hello, world! " + std::to_string(count_++);
      RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
      publisher_->publish(message);
    }
    rclcpp::TimerBase::SharedPtr timer_;
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
    size_t count_;
};

int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<MinimalPublisher>());
  rclcpp::shutdown();
  return 0;
}
```

this-> **create_publisher<std_msgs::msg::String>("topic", 10)**. creates a publisher of message type String from std_msgs, of name "topic" and the message will be publishes 10 times in a ms.

**publisher-> publish(message):** The command that publishes the data into the topic

these lines help the program run in a loop. Specifically, **rclcpp::spin** ensures of that. Once the program is completed the node destroys itself and then shutsdown.

# CREATING A CPP SUBSCRIBER

- Once you have publishes the data, you will also require a node that subscribes to this certain information.

Make sure to include rclcpp and the header that was used for publishing the data



```cpp
#include <memory>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
using std::placeholders::_1;

class MinimalSubscriber : public rclcpp::Node
{
  public:
    MinimalSubscriber()
    : Node("minimal_subscriber")
    {
      subscription_ = this->create_subscription<std_msgs::msg::String>(
      "topic", 10, std::bind(&MinimalSubscriber::topic_callback, this, _1));
    }

  private:
    void topic_callback(const std_msgs::msg::String & msg) const
    {
      RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg.data.c_str());
    }
    rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<MinimalSubscriber>());
  rclcpp::shutdown();
  return 0;
}
```

this-> **create_subscription<std_msgs::msg::String>(“topic”, 10, std::bind(&MinimalSubscriber::topic_callback, this, _1)**.
creates a subscriber of message type String from std_msgs, takes data from the topic named in the publisher “topic” in this case. **topic_callback** is the function receiving the data from the publisher

Similar to the publisher.
The lines in the main does exactly discussed previously

data being read from the string “msg” published over the topic

# MAKING YOUR NODES VISIBLE TO ROS2

- Step 1: **Add dependencies.** Remember package.xml ? Open the package xml, and make sure to fill the **<description>, <maintainer>, <license>** tag. Then, add a new line after the **ament_cmake** buildtool dependency and paste the following dependencies corresponding to your node's include statements:

  **<depend>rclcpp</depend>**

  **<depend>std_msgs</depend>**

  This declares the package needs rclcpp and std_msgs to build and execute the nodes.

- Step 2: **CmakeLists.txt.** Add the following lines below **find_package(ament_cmake REQUIRED):**

  **find_package(rclcpp REQUIRED)**

  **find_package(std_msgs REQUIRED)**

  Then, add the executable and name it something let us say **talker.**

  **add_executable(talker src/publisher_member_function.cpp)**

  **ament_target_dependencies(talker rclcpp std_msgs)**

  Finally add install(TARGETS...) so that ros2 run can find your executable:

  **install(TARGETS**

  **talker**

  **DESTINATION lib/${PROJECT_NAME})**

# MAKING YOUR NODES VISIBLE TO ROS2

```
cmake_minimum_required(VERSION 3.5)
project(cpp_pubsub)

# Default to C++14
if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 14)
endif()

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)

install(TARGETS
  talker
  DESTINATION lib/${PROJECT_NAME})

ament_package()
```

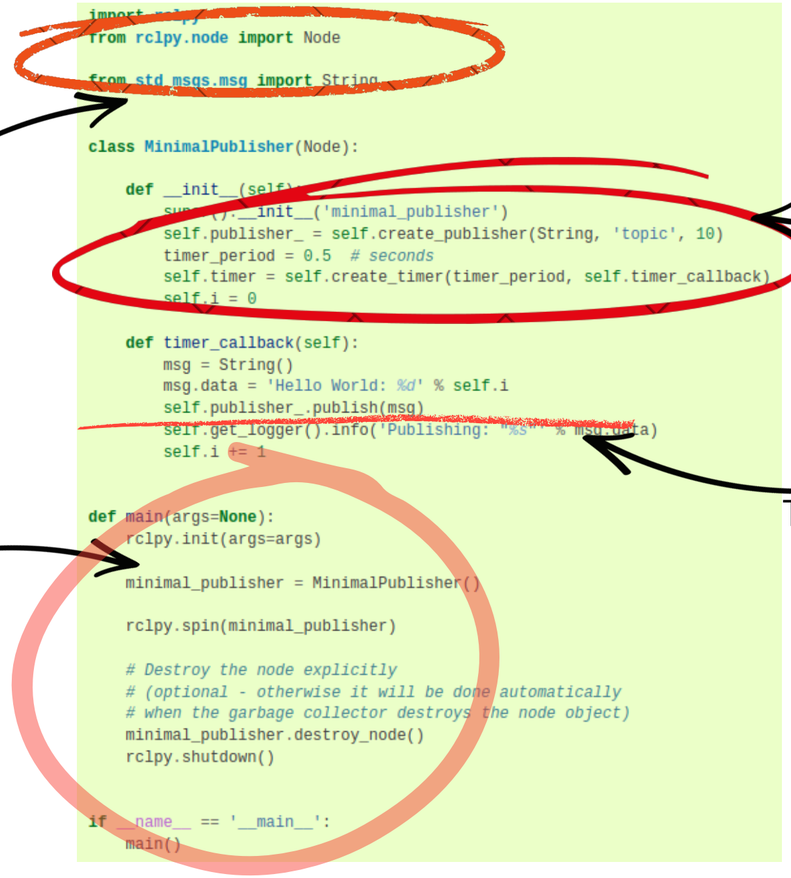
Here is the sample CMakeLists.txt for publisher try adding the subscriber executable in it.

Can you figure out how will you modify this .txt file for subscriber node?

# CREATING A PYTHON PUBLISHER

- Let us repeat the same exercise for python.

Headers: **rclpy** uses most common pieces for ROS2 python, **from std_msgs.msgs import String** includes the built-in message type you will use to publish data.



```
import rclpy
from rclpy.node import Node

from std_msgs.msg import String


class MinimalPublisher(Node):

    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, 'topic', 10)
        timer_period = 0.5  # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

    def timer_callback(self):
        msg = String()
        msg.data = 'Hello World: %d' % self.i
        self.publisher_.publish(msg)
        self.get_logger().info('Publishing: "%s"' % msg.data)
        self.i += 1


def main(args=None):
    rclpy.init(args=args)

    minimal_publisher = MinimalPublisher()

    rclpy.spin(minimal_publisher)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    minimal_publisher.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

**create_publisher(String, ‘topic’, 10)**. creates a publisher of message type String from std_msgs, of name “topic” and the message will be publishes 10 times in a ms.

**self.publisher_.publish(msg)**
The command that publishes the data into the topic

these lines help the program run in a loop. Specifically, **rclpy.spin** ensures of that. Once the program is completed the node destroys itself and then shutsdown.

# CREATING A PYTHON SUBSCRIBER

- Let us also look for the python subscriber

Make sure to include rclpy and the header that was used for publishing the data



```python
import rclpy
from rclpy.node import Node

from std_msgs.msg import String


class MinimalSubscriber(Node):

    def __init__(self):
        super().__init__('minimal_subscriber')
        self.subscription = self.create_subscription(
            String,
            'topic',
            self.listener_callback,
            10)
        self.subscription  # prevent unused variable warning

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)


def main(args=None):
    rclpy.init(args=args)

    minimal_subscriber = MinimalSubscriber()

    rclpy.spin(minimal_subscriber)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    minimal_subscriber.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

this-> **create_subscription(String, 'topic', self.listener_callback, 10)**.
creates a subscriber of message type String from std_msgs, takes data from the topic named in the publisher "topic" in this case. **listener_callback** is the function receiving the data from the publisher

Similar to the publisher.
The lines in the main does exactly discussed previously

data being read from the string "msg" published over the topic

# MAKING YOUR NODES VISIBLE TO ROS2

- Step 1: **Add dependencies.** Remember package.xml ? Open the package xml, and make sure to fill the **<description>, <maintainer>, <license>** tag. Then, add a new line after the **ament_python** buildtool dependency and paste the following dependencies corresponding to your node's include statements:

  **<depend>rclpy</depend>**

  **<depend>std_msgs</depend>**

  This declares the package needs rclcpp and std_msgs to build and execute the nodes.

- Step 2: **setup.py.** First, make sure to match the maintainer details, description and license. Then, add the following lines within the **console_scripts** of the **entry_points** field:

```
entry_points={
        'console_scripts': [
        'talker = py_pubsub.publisher_member_function:main',
        'listener = py_pubsub.subscriber_member_function:main',
         ],},
```

  Don't forget to save before doing colcon build.

# OPTIONAL ASSIGNMENT FOR PRACTICE

- Make a simple publisher in both cpp and python that publishes velocity “cmd_vel” for turtlesim turlte.
- To make your programming skills sharp you can try making the turtle move in a square by giving a series of cmd_vel values.

# THANK YOU

**FOR YOUR TIME AND ATTENTION TODAY. WE APPRECIATE YOUR ENGAGEMENT.**




9812930249

aayush25263101
@iitgoa.ac.in

LAB-4, ACEDAMIC BLOCK-1